Benchmark Comparativo entre Ariel OS e FreeRTOS em Sistemas de Tempo Real Restritos

Experiência Criativa: Projeto Transformador II
Bacharelado em Ciência da Computação – PUCPR

Turma A – Equipe 4
Bruno Moschetta, Daniel Lima, Isabella Conceição, Vinícius Pan
bruno.moschetta, l.pereira15, isabella.lucena, vinicius.pan@pucpr.edu.br

I. Descrição do Projeto

O crescimento acelerado de sistemas embarcados e dispositivos de Internet das Coisas (IoT) tem impulsionado a adoção de sistemas operacionais de tempo real (RTOS) capazes de operar sob fortes restrições de memória, energia e previsibilidade temporal em microcontroladores de 32 bits [1]. Nesse cenário, o FreeRTOS se consolidou como o RTOS de referência na indústria, sendo amplamente utilizado por sua maturidade, leveza e suporte extenso a arquiteturas de hardware. Entretanto, por ser implementado em C, herda as fragilidades clássicas dessa linguagem quanto ao gerenciamento manual de memória, como evidenciado por ataques de buffer overflow (return-to-libc e injeção de código) bem-sucedidos mesmo diante dos mecanismos de proteção disponíveis no FreeRTOS [2] e por estudos de injeção de falhas que mostram a sensibilidade de suas estruturas internas (TCBs, filas e variáveis globais) a corrupções que comprometem a integridade do sistema [3].

Paralelamente, a linguagem Rust vem se consolidando como alternativa para software de sistemas embarcados, combinando controle de baixo nível com segurança de memória garantida em tempo de compilação por meio de seu modelo de posse (*ownership*). Essas características já motivaram o surgimento de diversos RTOS e frameworks de concorrência embarcados em Rust, como RTIC, Embassy e o kernel de tempo real HarSaRK-RS, que reporta reduções expressivas de tempo de *context switch* frente a kernels equivalentes em C [4]. Nesse contexto, surge o Ariel OS, um RTOS recente, open source e escrito inteiramente em Rust, que introduz suporte a escalonamento preemptivo multicore combinado à programação assíncrona sobre arquiteturas ARM Cortex-M, RISC-V e Xtensa [1].

Embora trabalhos recentes já tenham comparado quantitativamente frameworks de concorrência em Rust e C, como a avaliação de FreeRTOS, RTIC e Embassy em métricas de latência, jitter e uso de memória [5], e estudos de caso industriais tenham indicado que Rust não impõe penalidades relevantes de desempenho ou footprint frente ao C [6], ainda há uma lacuna de avaliações empíricas comparando diretamente o Ariel OS ao FreeRTOS sob métricas de desempenho padronizadas.

Este artigo apresenta uma avaliação experimental comparativa do FreeRTOS e do Ariel OS na plataforma ESP32-WROOM-32, buscando identificar os *trade-offs* entre os dois sistemas quanto ao uso de memória, desempenho e previsibilidade temporal.

Para isso, foram implementadas três aplicações de complexidade crescente: Blinky (simples), UART-to-LED (intermediária) e IoT (complexa). Essas aplicações foram utilizadas para avaliar o footprint de memória Flash e RAM, o *jitter* periódico, a latência de interrupção e a latência de *task-switch*, permitindo caracterizar as diferenças observadas entre os dois sistemas operacionais sob cargas de trabalho equivalentes.

II. Materiais e Métodos

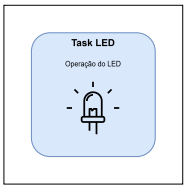
A avaliação experimental comparou o FreeRTOS, baseado em C, e o Ariel OS, desenvolvido em Rust, utilizando cargas de trabalho equivalentes executadas sobre a mesma plataforma de hardware. Foi utilizada a ESP32-WROOM-32, baseada na arquitetura Xtensa, mantida fixa durante a avaliação. Trabalhos anteriores demonstram a execução do Ariel OS em diferentes arquiteturas de microcontroladores [1], incluindo plataformas da família ESP.

Foram implementadas três aplicações de complexidade crescente, cuja estrutura é apresentada na Fig. 1. A primeira, *Blinky* (Fig. 1a), consiste em uma única tarefa periódica responsável por alternar o estado de um LED. A segunda, *UART-to-LED* (Fig. 1b), realiza a recepção de dados pela interface UART e a sinalização da tarefa responsável pela alteração do LED. Por fim, a aplicação IoT (Fig. 1c) integra a aquisição sintética de dados de um sensor, processamento das leituras, atuação e registro periódico de informações.

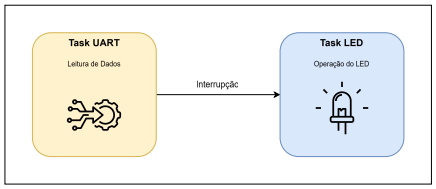
As implementações empregam os mecanismos próprios de cada RTOS, preservando a mesma estrutura funcional e os parâmetros de execução definidos para cada aplicação.

As métricas avaliadas foram footprint de memória Flash e RAM, jitter periódico, latência de interrupção e latência de troca de tarefa (task-switch), as mesmas empregadas em avaliações recentes de frameworks de concorrência para sistemas embarcados [5]. A forma de obtenção de cada métrica é detalhada ao final desta seção.

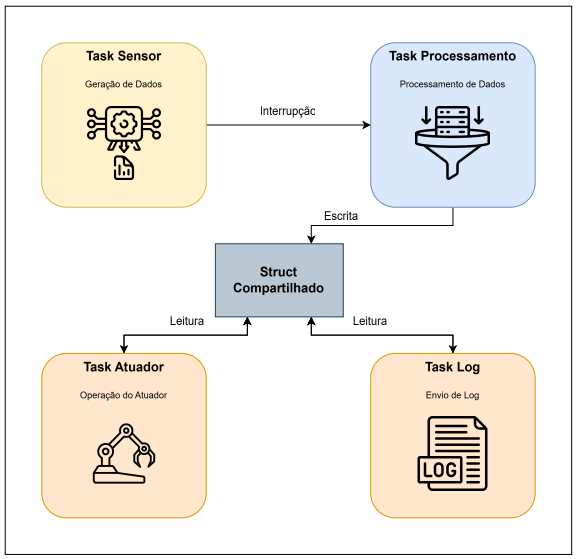
Os experimentos foram conduzidos em uma única placa ESP32-WROOM-32 (Xtensa LX6), com frequência de CPU fixa em 240 MHz e execução em um único núcleo nos



(a) Blinky



(b) UART-to-LED



(c) Aplicação IoT

Figura 1: Estrutura das aplicações utilizadas na avaliação experimental.

dois sistemas. O FreeRTOS foi compilado com o ESP-IDF (tick de 1 ms, otimização de desempenho e task watchdog desabilitado). O Ariel OS 0.6.0 foi compilado com o esp-hal 1.0 e threads preemptivas. Cada aplicação foi implementada nos dois sistemas com a mesma estrutura de tarefas, as mesmas prioridades e a mesma carga de trabalho.

O Blinky consiste em uma única tarefa que alterna um LED com períodos de 1, 10 e 100 ms, ativada por prazo absoluto. O UART-to-LED usa uma tarefa de recepção serial que encaminha cada comando, por fila (FreeRTOS) ou canal (Ariel OS), a uma tarefa de maior prioridade responsável pelo LED. Antes de cada encaminhamento, a tarefa de recepção dispara a mesma interrupção de software usada na aplicação IoT e aguarda o fim do seu tratamento, de modo que a latência de interrupção também é medida sem sobrepor a janela de *task-switch*. A aplicação IoT reproduz a estrutura da Fig. 1c. Uma tarefa de aquisição, ativada a cada 10 ms, gera uma leitura sintética determinística de 12 bits e dispara uma interrupção de software que faz o papel do sinal *data-ready* de um sensor. A rotina de interrupção sinaliza a tarefa de processamento, que aplica média móvel de oito amostras, conversão para milivolts e alarme com histerese, e publica o resultado em uma estrutura compartilhada lida pelas tarefas de atuação (20 ms) e de log (1 s).

Todos os instantes foram obtidos pelo contador de ciclos da CPU (registrador CCOUNT), equivalente ao contador DWT usado por Wang [5], o que dá resolução de aproximadamente 4,17 ns. A cada execução, esse contador é conferido contra um temporizador independente de 1 µs ao longo de 100 ms. O jitter periódico foi calculado como $J_i = (C_i - C_{i-1}) - C_{esperado}$, a partir do instante logo após cada ativação. A latência de interrupção corresponde ao intervalo entre o disparo da interrupção de software e a primeira instrução da rotina de tratamento. A latência de task-switch corresponde ao intervalo entre a sinalização de uma tarefa e a primeira instrução executada por ela após despertar.

Seguindo o protocolo de Wang [5], cada métrica reúne 1005 amostras, das quais as cinco primeiras são descartadas como aquecimento. As amostras são armazenadas em memória estática durante a janela de medição e exportadas pela serial somente ao final, para que a exportação não interfira nos tempos medidos. A coleta é automatizada por um script que reinicia a placa entre execuções e rejeita registros incompletos, corrompidos ou com eventos perdidos. Para cada execução, são calculados mínimo, máximo, média, desvio-padrão e percentis P50, P90, P95, P99 e P99,9. Os resultados das execuções são então agregados para avaliar também a variação entre execuções. O footprint de memória Flash foi obtido a partir das seções dos binários ELF gerados para cada aplicação.

O projeto não utilizou um conjunto de dados externo. Os dados experimentais foram produzidos pelas próprias execuções dos *benchmarks*, sendo compostos pelas medições obtidas para cada RTOS e aplicação. As versões dos sistemas operacionais, compiladores, parâmetros de compilação e configurações utilizadas foram registradas para garantir a reprodutibilidade dos experimentos.

O código-fonte e os artefatos utilizados estão disponíveis no repositório público do projeto em https://github.com/Vinokaa/FreeRTOS-ArielOS-Benchmark.

A Tabela I resume os principais materiais, aplicações, métricas e procedimentos adotados na avaliação experimental, enquanto a Tabela II apresenta especificações das ferramentas utilizadas.

Tabela I: Resumo dos materiais e métodos

Item	Descrição
RTOS	FreeRTOS e Ariel OS
Hardware	ESP32-WROOM-32 (Xtensa)
Aplicações	Blinky, UART-to-LED e aplicação IoT
Métricas	Flash, RAM, jitter, latência de interrupção e task-switch
Análise	Avaliação experimental de desempenho e previsibilidade temporal

III. Estágios do Projeto e Marcos Observáveis

O desenvolvimento foi estruturado em seis etapas progressivas, abrangendo a preparação da plataforma, a implementação das aplicações, a instrumentação, a coleta e a análise dos resultados. As atividades foram organizadas de modo a permitir o desenvolvimento paralelo das implementações nos dois sistemas operacionais.

Tabela II: Configuração de compilação dos sistemas avaliados

Parâmetro	FreeRTOS (C)	Ariel OS (Rust)
Sistema	ESP-IDF 6.2.0-dev	Ariel OS 0.6.0 (`esp-hal` 1.0.0)
Kernel	FreeRTOS 10.5.1	`ariel-os-threads` 0.6.0
Compilador	GCC 16.1.0	`rustc` 1.97.0-nightly
Backend	GCC	LLVM 21.1.3 (Xtensa)
Otimização	`-O2`	`opt-level=2`
Núcleo único	`FREERTOS_UNICORE=y`	Feature `single-core`

1) Validação da plataforma e implementação do Blinky Inicialmente, foram configuradas e validadas as toolchains necessárias para FreeRTOS e Ariel OS na ESP32-WROOM-32. A aplicação *Blinky* foi implementada em conjunto nos dois ambientes, servindo como teste inicial de compilação, gravação, execução e acesso aos periféricos. Essa etapa também permitiu a familiarização dos integrantes com a estrutura dos dois RTOS.

2) Implementação do UART-to-LED Após validação inicial, foi desenvolvida a aplicação *UART-to-LED*. Cada integrante assumiu inicialmente a implementação em um dos RTOS, preservando a mesma funcionalidade e os mesmos parâmetros de execução. A implementação permitiu validar a recepção de comandos pela UART e a sinalização da tarefa responsável pela alteração do LED.

3) Implementação da aplicação IoT Foi implementada a aplicação de maior complexidade, composta por aquisição sintética de dados de sensor, processamento, atuação e registro periódico de informações. As versões em FreeRTOS e Ariel OS preservaram a mesma estrutura funcional e parâmetros equivalentes de execução.

4) Instrumentação e validação das medições Foram incorporados mecanismos para obtenção das métricas de Flash, RAM, *jitter*, latência de interrupção e *task-switch*. Também foram realizadas execuções piloto para validar o procedimento de coleta, o formato dos dados e a consistência das medições.

5) Execução e análise dos experimentos Os *benchmarks* foram executados sob configurações equivalentes de software e hardware. As medições foram armazenadas em arquivos e analisadas por meio das estatísticas definidas na Seção II. Os resultados obtidos foram organizados em tabelas e gráficos comparativos.

6) Consolidação dos resultados Os resultados experimentais foram organizados de forma a evidenciar os *trade-offs* entre FreeRTOS e Ariel OS. O código-fonte, os registros experimentais e os resultados das análises compõem os artefatos do projeto. A apresentação dos resultados na Poster Session permanece prevista para 17/11.

IV. Cronograma

O cronograma do projeto foi organizado de forma progressiva, acompanhando as etapas de desenvolvimento definidas na Seção III. As atividades iniciais, realizadas durante o período de planejamento, compreenderam a revisão bibliográfica, a definição do escopo e a familiarização com as linguagens, os RTOS e as ferramentas de desenvolvimento.

As etapas seguintes contemplaram a validação da plataforma, a implementação das aplicações Blinky, UART-to-LED e IoT, a instrumentação das métricas e a coleta dos resultados experimentais. As atividades foram distribuídas considerando a possibilidade de desenvolvimento paralelo entre os integrantes.

A Tabela III apresenta a distribuição das atividades ao longo do semestre, incluindo os principais marcos estabelecidos no plano de ensino: entrega da implementação e dos resultados experimentais em 31/10, entrega do artigo científico e pôster em 09/11 e apresentação na Poster Session, prevista para 17/11.

Tabela III: Cronograma de desenvolvimento do projeto.

Atividade	04/08–17/08	18/08–31/08	01/09–14/09	15/09–28/09	29/09–12/10	13/10–31/10	01/11–09/11	10/11–17/11
Revisão bibliográfica e definição do escopo	X	X						
Aprendizagem das linguagens, RTOS e dinâmica de desenvolvimento	X	X						
Validação da plataforma e Blinky			X					
Implementação do UART-to-LED			X	X				
Implementação da aplicação IoT				X	X			
Instrumentação e execuções piloto					X	X		
Execução e análise dos experimentos						X	X	
Consolidação dos resultados							X	
Artigo científico e pôster						X	X	
Preparação para a Poster Session								X

V. Resultados

A. Blinky

O footprint do Blinky (Tabela IV) serve de referência para as demais aplicações, já que ele tem uma única tarefa e quase todo o binário corresponde à base de cada sistema. O Ariel OS ocupa 62.047 bytes de Flash, cerca de 58% dos 107.691 bytes do FreeRTOS. A diferença não está no código executado a partir da Flash, que é próximo nos dois sistemas (37.555 contra 40.786 bytes). Ela está no código em IRAM: o ESP-IDF mantém 34.691 bytes do kernel e das rotinas de interrupção em IRAM, contra apenas 6.112 bytes no Ariel OS. Os dados somente leitura do Ariel OS também ocupam menos da metade (11.396 contra 24.568 bytes). Na RAM estática, o Ariel OS soma 301.488 bytes contra 48.396 bytes, mas quase todo esse valor vem da seção ZI (288.392 bytes). Essa seção fica praticamente no mesmo tamanho nas três aplicações (entre 287.952 e 288.392 bytes), independentemente do número de tarefas, o que confirma que é uma reserva fixa do sistema, que ocupa praticamente toda a DRAM interna livre, e não um consumo da aplicação. No FreeRTOS, a maior parcela da RAM estática é o próprio código em IRAM, e suas pilhas e TCBs, alocadas no heap em tempo de execução, não entram nessa conta. Por isso, como discutido na Seção V-C, a RAM estática não permite uma comparação direta entre os dois sistemas.

Na Tabela V, o FreeRTOS apresenta jitter nulo nos três períodos, comportamento também observado por Wang [5]. No Xtensa, o tick do escalonador é gerado pelo comparador CCOMPARE0, que é comparado ao próprio CCOUNT, o

Tabela IV: Footprint de memória da aplicação Blinky (bytes)

Métrica	FreeRTOS	Ariel OS
Code	75477	43667
RO data	24568	11396
RW data	7646	6984
ZI data	6059	288392
Flash total	107691	62047
RAM estática	48396	301488
Código em flash	40786	37555
Código em IRAM	34691	6112

mesmo contador usado nas medições. Cada tick ocorre, portanto, em um múltiplo exato de 240.000 ciclos, e a diferença $C_i - C_{i-1}$ coincide sempre com o período esperado. O jitter nulo é real em relação ao clock da CPU, mas decorre de o escalonador e a medição compartilharem a mesma base de tempo, o que não ocorre no Ariel OS.

No Ariel OS (Tabela VI), o jitter decorre da quantização da base de tempo, e não de imprevisibilidade. Os valores observados ficam em poucos níveis discretos, sempre separados por cerca de 240 ciclos, ou seja, 1 μs a 240 MHz. O temporizador do Ariel (pilha embassy-time sobre um timer de periférico do esp-hal) roda num domínio de clock separado, com resolução de cerca de 1 μs. O padrão é um batimento de fase entre esse timer e o CCOUNT. Por isso, o desvio-padrão de cerca de 180 ciclos não deve ser interpretado como variabilidade do escalonador.

Tabela V: *Jitter* da tarefa periódica do Blinky no FreeRTOS, em ciclos de CPU.

Período	Min	Max	Média	Mediana	DP	P90	P95	P99	P99.9
1 ms	0	0	0	0	0	0	0	0	0
10 ms	0	0	0	0	0	0	0	0	0
100 ms	0	0	0	0	0	0	0	0	0

Tabela VI: *Jitter* da tarefa periódica do Blinky no Ariel OS, em ciclos de CPU.

Período	Min	Max	Média	Mediana	DP	P90	P95	P99	P99.9
1 ms	−255	360	0,126	−15	180,621	120	360	360	360
10 ms	−165	228	−0,011	−15	76,931	87	87	228	228
100 ms	−165	228	0,017	−15	76,983	87	87	228	228

B. UART-to-LED

Na aplicação *UART-to-LED*, o script de coleta envia 1005 comandos por execução, um a cada 500 ms, e cada comando gera uma amostra de latência de interrupção e uma de *task-switch*. Foram realizadas cinco execuções por sistema, o que totaliza 5000 amostras por métrica após o descarte do aquecimento. Os resultados estão nas Tabelas VII e VIII.

Tabela VII: Latência de *task-switch* (tarefa → tarefa) (UART-to-LED), em ciclos de CPU.

Sistema	Min	Max	Média	Mediana	DP	P90	P95	P99	P99.9
FreeRTOS	1586	2516	1590,294	1586	55,205	1586	1586	1586	2477
Ariel OS	1566	1566	1566	1566	0	1566	1566	1566	1566

Tabela VIII: Latência de interrupção (UART-to-LED), em ciclos de CPU.

Sistema	Min	Max	Média	Mediana	DP	P90	P95	P99	P99.9
FreeRTOS	283	770	283,583	283	16,816	283	283	283	768
Ariel OS	626	626	626	626	0	626	626	626	626

A latência de *task-switch* (Tabela VII) é praticamente a mesma nos dois sistemas: 1566 ciclos no Ariel OS e mediana de 1586 ciclos no FreeRTOS, uma diferença de 20 ciclos (cerca de 83 ns). O que os distingue é a dispersão. No Ariel OS, as 5000 amostras tiveram exatamente o mesmo valor em todas as execuções. No FreeRTOS, 4965 amostras ficaram em 1586 ciclos e 35 (0,7%) ficaram entre 1638 e 2516 ciclos, o que eleva a média para 1590,3 ciclos e o desvio-padrão para 55,2 ciclos.

A latência de interrupção (Tabela VIII) repete o padrão da aplicação IoT (Tabela XII): o FreeRTOS responde em 283 ciclos e o Ariel OS em 626 ciclos, cerca de 2,2 vezes mais. Como a interrupção de software é disparada da mesma forma nas duas aplicações, a proximidade com os valores do IoT (274 e 636 ciclos) indica que esse custo pertence ao caminho de entrada de interrupção de cada sistema, e não à carga da aplicação. De novo, o Ariel OS não apresentou variação, enquanto o FreeRTOS teve seis amostras (0,12%) entre 768 e 770 ciclos.

Os valores atípicos do FreeRTOS são compatíveis com a interrupção periódica do *tick* de 1 ms caindo dentro da janela medida. Como os comandos chegam sem sincronia com o *tick*, a probabilidade de coincidência é a razão entre a duração da janela e o período do *tick* (240.000 ciclos): cerca de 0,66% para o *task-switch* e 0,12% para a interrupção, valores próximos às frequências observadas (0,70% e 0,12%). No Ariel OS, a ausência de valores atípicos em 5000 amostras indica que nenhuma interrupção periódica atingiu as janelas de medição.

Tabela IX: Footprint de memória da aplicação UART-to-LED (bytes)

Métrica	FreeRTOS	Ariel OS
Code	92785	58459
RO data	30092	13432
RW data	7994	7424
ZI data	10163	287952
Flash total	130871	79315
RAM estática	57156	301696
Código em flash	53786	52139
Código em IRAM	38999	6320

O *footprint* (Tabela IX) segue o padrão observado no Blinky. O binário do Ariel OS ocupa 79.315 bytes de Flash, cerca de 61% dos 130.871 bytes do FreeRTOS, e 301.696 bytes de RAM estática, contra 57.156 bytes. Quase toda essa RAM está na seção ZI (287.952 bytes), que mantém o tamanho já observado no Blinky (Seção V-A).

C. Aplicação IoT

O footprint do IoT (Tabela X) repete o padrão das outras aplicações. O Ariel OS ocupa 66.219 bytes de Flash, 60% dos 110.167 bytes do FreeRTOS. Do Blinky ao IoT, o binário cresce apenas 2.476 bytes no FreeRTOS e 4.172 bytes no Ariel OS, então nos dois casos a base do sistema domina o tamanho final. Na RAM estática, o Ariel OS soma 301.708 bytes contra 60.944 bytes.

Essa comparação, porém, não reflete o uso real de memória, por dois motivos. Primeiro, a seção ZI do Ariel OS é a reserva fixa descrita na Seção V-A, e não um consumo da aplicação. Segundo, o método de medição conta as pilhas das threads do Ariel OS, que são estáticas, mas não conta as pilhas e TCBs do FreeRTOS, alocadas no heap em tempo de execução. Assim, a RAM estática subestima o uso real do FreeRTOS e superestima o espaço de fato ocupado pelo Ariel OS. A comparação justa exige medir em tempo de execução o heap livre e o pico de uso das pilhas nos dois sistemas.

Tabela X: Footprint de memória da aplicação IoT (bytes)

Métrica	FreeRTOS	Ariel OS
Code	77441	47663
RO data	25080	11552
RW data	7646	7004
ZI data	18123	288368
Flash total	110167	66219
RAM estática	60944	301708
Código em flash	42266	41327
Código em IRAM	35175	6336

O jitter da tarefa de aquisição (Tabela XI e Fig. 2) não é aleatório nos dois sistemas. No FreeRTOS, 98% das 5000 amostras valem exatamente ± 157 ciclos, metade de cada sinal, e os 2% restantes valem ± 301 ciclos. No Ariel OS, as amostras se dividem igualmente entre um grupo negativo e um positivo, quase todas entre 750 e 1250 ciclos em módulo, com cerca de 1% chegando a ± 1353 e ± 1368 ciclos. A causa é a coincidência de prazos entre as tarefas. A tarefa de atuação (20 ms) vence no mesmo instante que a de aquisição a cada duas ativações, e a de log (1 s) a cada cem, ou seja, dez vezes em cada janela de 1000 amostras. Embora a aquisição tenha a maior prioridade, o tratamento do temporizador precisa liberar todas as tarefas vencidas antes que ela execute. A ativação que coincide é atrasada em um valor d, e como o jitter é a diferença entre ativações consecutivas, a amostra seguinte aparece adiantada pelo mesmo d. Isso produz a sequência alternada +d, -d e uma distribuição simétrica de média zero. Pelo mesmo motivo, a mediana de 510 ciclos do Ariel OS cai no intervalo entre os dois grupos e não representa nenhuma amostra típica.

Dessa forma, a amplitude da distribuição mede o custo de liberar uma tarefa a mais no mesmo instante. No FreeRTOS, esse custo é de 157 ciclos (0,65 µs). No Ariel OS, fica entre aproximadamente 750 e 1250 ciclos (3 a 5 µs), de cinco a oito vezes mais. O custo da terceira tarefa coincidente, o log, é praticamente igual nos dois sistemas: 301 - 157 = 144 ciclos no FreeRTOS e 1368 - 1227 = 141 ciclos no Ariel OS. Assim, a diferença se concentra na primeira expiração adicional, e não na liberação de cada tarefa. Uma hipótese compatível é um custo fixo no tratamento de múltiplas expirações pela pilha de tempo do Ariel OS, como reprogramar o alarme. Dentro de cada grupo, os valores do Ariel OS aparecem separados por cerca de 240 ciclos (por exemplo, 987 e 1227, ou -972 e -1212), o que repete a quantização de 1 µs do temporizador já observada no Blinky. Esse comportamento é coerente com Wang [5], cuja aplicação LPV também produziu jitter simétrico e centrado em zero no FreeRTOS (± 335 ciclos) e no Embassy (± 370 ciclos). Esse resultado também responde à RQ3 de Wang sobre a origem dos trade-offs: o desvio-padrão 6,4 vezes maior do Ariel OS não indica um escalonador imprevisível. Ele é um custo determinístico e repetível, ligado à coincidência entre períodos.

A latência de interrupção (Tabela XII) foi constante nos dois sistemas em todas as amostras: 274 ciclos (1,14 µs) no FreeRTOS e 636 ciclos (2,65 µs) no Ariel OS, 2,3 vezes mais. Os valores são próximos dos obtidos no UART-to-LED (283 e 626 ciclos), o que confirma que o custo é do caminho de entrada de interrupção de cada sistema. Os dois valores são muito maiores que os 40 a 47 ciclos registrados por Wang [5] num Cortex-M4. Nessa arquitetura, o controlador NVIC empilha o contexto em hardware. No Xtensa LX6, o salvamento de contexto e o despacho até a rotina da aplicação são feitos em software. O ESP-IDF usa vetores em assembly e mantém o kernel e as rotinas de interrupção em IRAM (35.175 bytes de código em IRAM, Tabela X). No Ariel OS, o despacho passa pela camada genérica do esp-hal, e apenas 6.336 bytes de código ficam em IRAM.

A latência de task-switch da interrupção até a tarefa de processamento (Tabela XIII) segue a mesma proporção: 820 ciclos (3,4 µs) no FreeRTOS e mediana de 1822 ciclos (7,6 µs) no Ariel OS, 2,2 vezes mais. No FreeRTOS, as 5000 amostras foram idênticas. Ao contrário do UART-to-LED, o tick de 1 ms nunca cai dentro da janela medida, porque a aquisição é ativada pelo próprio tick e a janela começa sempre na mesma fase. No Ariel OS, exatamente 50 amostras (1%, dez por execução) ficaram em 2767 ou 2768 ciclos, cerca de 946 ciclos acima do valor típico. A frequência coincide com a da tarefa de log, o que indica de novo a coincidência de prazos como causa. Comparado com o UART-to-LED, em que os dois sistemas empatam na sinalização entre tarefas, esse resultado localiza o custo adicional do Ariel OS na transição da interrupção para o escalonador. Também é preciso considerar que os primitivos não são idênticos. O FreeRTOS usa notificação direta à tarefa (vTaskNotifyGiveFromISR), o mecanismo mais leve do kernel, e o Ariel OS usa thread flags. Wang [5] já aponta que a escolha do primitivo de comunicação afeta o valor medido.

Tabela XI: *Jitter* da tarefa periódica do sensor de 10 ms (IoT), em ciclos de CPU.

Sistema	Min	Max	Média	Mediana	DP	P90	P95	P99	P99.9
FreeRTOS	−301	301	0	0	161,162	157	157	158,44	301
Ariel OS	−1353	1368	−0,120	510	1029,464	1212	1227	1263	1368

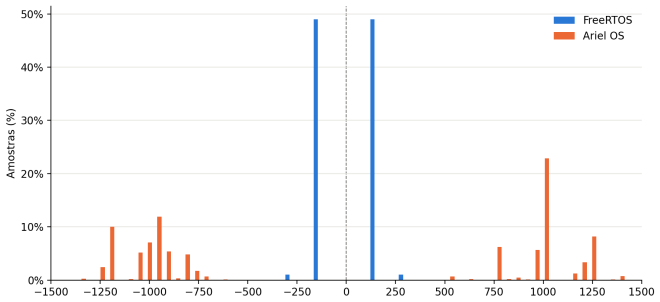


Figura 2: Distribuição do jitter da tarefa de aquisição de 10 ms (aplicação IoT) em ciclos de CPU (faixas de 48 ciclos; 240 ciclos = 1 µs).

Tabela XII: Latência de interrupção (IoT), em ciclos de CPU.

Sistema	Min	Max	Média	Mediana	DP	P90	P95	P99	P99.9
FreeRTOS	274	274	274	274	0	274	274	274	274
Ariel OS	636	636	636	636	0	636	636	636	636

Tabela XIII: Latência de *task-switch* (ISR → tarefa) (IoT), em ciclos de CPU.

Sistema	Min	Max	Média	Mediana	DP	P90	P95	P99	P99.9
FreeRTOS	820	820	820	820	0	820	820	820	820
Ariel OS	1822	2768	1831,456	1822	94,095	1822	1822	1831,45	2768

VI. Conclusão

Este trabalho comparou o Ariel OS e o FreeRTOS na ESP32-WROOM-32 com três aplicações de complexidade crescente, seguindo o protocolo de Wang [5] com o contador de ciclos CCOUNT, 1000 amostras válidas por métrica e cinco execuções por sistema. Nenhum dos dois foi superior em todas as métricas. O FreeRTOS foi mais rápido no caminho de interrupção: latência de interrupção 2,2 a 2,3 vezes menor, task-switch a partir de uma interrupção 2,2 vezes menor e jitter de ativação com amplitude de cinco a oito vezes menor quando há tarefas com prazos coincidentes. O Ariel OS gerou binários com 58% a 61% da Flash do FreeRTOS e teve latência de task-switch entre tarefas equivalente à do FreeRTOS (1566 contra 1586 ciclos), sem nenhuma variação nas amostras.

O principal resultado é que as diferenças de tempo observadas são, em sua maioria, determinísticas. As latências do Ariel OS foram praticamente constantes, e quase toda a variabilidade dos dois sistemas tem causa identificável: a quantização de 1 µs do temporizador do Ariel OS, o tick de 1 ms do FreeRTOS caindo dentro da janela medida, ou a coincidência periódica entre os prazos das tarefas de atuação e log. Dessa forma, desvio-padrão maior não significa imprevisibilidade. Como em Wang [5], a desvantagem do sistema em Rust é um custo fixo e mensurável, não um comportamento errático. Os resultados também localizam esse custo: ele está na entrada e saída de interrupções e no tratamento do temporizador, não na troca de contexto entre tarefas. Isso aponta onde o Ariel OS pode evoluir, por exemplo com rotinas críticas em IRAM e um despacho de interrupções mais direto no esp-hal.

Do ponto de vista qualitativo, os custos medidos, de alguns microssegundos, são compatíveis com os requisitos de grande parte das aplicações IoT. Em troca, o Ariel OS elimina por construção, no código seguro (safe Rust), as classes de falha que motivaram este estudo, estouros de buffer e corrupção de estruturas do kernel, demonstradas no FreeRTOS por Mullen e Meany [2] e Bosio et al. [3], restringindo-as aos trechos unsafe. Portanto, o Ariel OS se mostra uma alternativa viável para aplicações de tempo real brando e de IoT em que segurança de memória e tamanho do binário pesam mais que microssegundos de latência. O FreeRTOS continua preferível quando a latência mínima de interrupção é o requisito dominante. Cabe lembrar que segurança de memória não equivale a certificação de segurança nem garante, por conta própria, a correção temporal [5].

Os resultados têm limitações que restringem sua generalização. Foram usados uma única placa, um único núcleo e cinco execuções por configuração. Os compiladores são diferentes (GCC no ESP-IDF e LLVM para Xtensa no Ariel OS), embora ambos tenham utilizado o mesmo perfil de otimização durante a compilação. Os primitivos de comunicação não são idênticos (notificação direta e fila no FreeRTOS, thread flags e canal no Ariel OS). A RAM estática não contabiliza da mesma forma as pilhas dos dois sistemas. Também não houve cenários de estresse nem validação externa por analisador lógico.

Como trabalhos futuros, propõem-se: medir em tempo de execução o uso de heap e pilhas, analisar o código desmontado para separar o custo do compilador do custo do sistema, validar as medições por GPIO como em Wang [5], avaliar o escalonamento multicore que o Ariel OS oferece e repetir os experimentos em outras arquiteturas suportadas por ele, como RISC-V e Cortex-M.

Referências

[1] E. Frank, K. Schleiser, R. Fouquet, K. Zandberg, C. Amsüss, and E. Baccelli, "Ariel OS: An embedded Rust operating system for networked sensors & multi-core microcontrollers," in *2025 21st International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT)*. IEEE, 2025, pp. 241–245.

[2] G. Mullen and L. Meany, "Assessment of buffer overflow based attacks on an IoT operating system," in *2019 Global IoT Summit (GIoTS)*. Aarhus, Denmark: IEEE, 2019, pp. 1–6.

[3] A. Bosio, M. Rebaudengo, and A. Savino, "Reliability assessment of FreeRTOS in embedded systems," in *2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks - Supplemental Volume (DSN-S)*. IEEE, 2022.

[4] N. Gupta, G. Banda, N. Hubballi, and K. V. Srinivas, "A hard real-time kernel for CPIoT systems with safety features in Rust," *IEEE Access*, vol. 13, pp. 116 803–116 820, 2025.

[5] G. Wang, "Concurrency frameworks for embedded systems: A comparative evaluation of FreeRTOS, RTIC, and Embassy," *IET Computers & Digital Techniques*, vol. 2026, p. 7881841, 2026.

[6] B. Thapa, D. Alfonso, L. Bini, L. Mapelli, K. Schleiser, R. Fouquet, and E. Baccelli, "Embedded Rust or C firmware? lessons from an industrial microcontroller use case with Ariel OS," in *Proceedings of IEEE DCOSS-IoT, Workshop on IoT Applications and Industry 5.0 (IoTI5)*. IEEE, June 2026.